Practical 6

Part A:

Aim:

1. Remote control: an interface Switchable with on()/off() and a default toggle(); Fan and Light implement it. Loop over a Switchable[] toggling each. Then add a functional interface that decides, given a device and an hour, whether it may switch on, and supply it once as an anonymous class and once as a lambda.

Code:

RemoteControl.java

```
public class RemoteControl {

    // Interface
    interface Switchable {

        void on();

        void off();

        // Default method
        default void toggle() {
            on();
        }
    }

    // Fan implements Switchable
    static class Fan implements Switchable {

        private boolean on = false;

        @Override
        public void on() {
            on = true;
            System.out.println("Fan is ON");
        }

        @Override
        public void off() {
            on = false;
            System.out.println("Fan is OFF");
        }

        public boolean isOn() {
```

```
            return on;
        }
    }

    // Light implements Switchable
    static class Light implements Switchable {

        private boolean on = false;

        @Override
        public void on() {
            on = true;
            System.out.println("Light is ON");
        }

        @Override
        public void off() {
            on = false;
            System.out.println("Light is OFF");
        }

        public boolean isOn() {
            return on;
        }
    }

    // Functional interface
    interface SwitchPolicy {

        boolean maySwitchOn(Switchable device, int hour);
    }

    public static void main(String[] args) {

        // Create devices
```

```
        Switchable[] devices = {
            new Fan(),
            new Light()
        };

        System.out.println("=== Toggle Devices ===");

        // Toggle every device
        for (Switchable device : devices) {
            device.toggle();
        }

        System.out.println();

        // Anonymous class implementation
        SwitchPolicy daytimePolicy = new SwitchPolicy() {

            @Override
            public boolean maySwitchOn(Switchable device, int
hour) {
                return hour >= 6 && hour <= 22;
            }
        };

        // Lambda implementation
        SwitchPolicy nightPolicy =
                (device, hour) -> hour >= 18 && hour <= 23;

        System.out.println("=== Switch Policy ===");

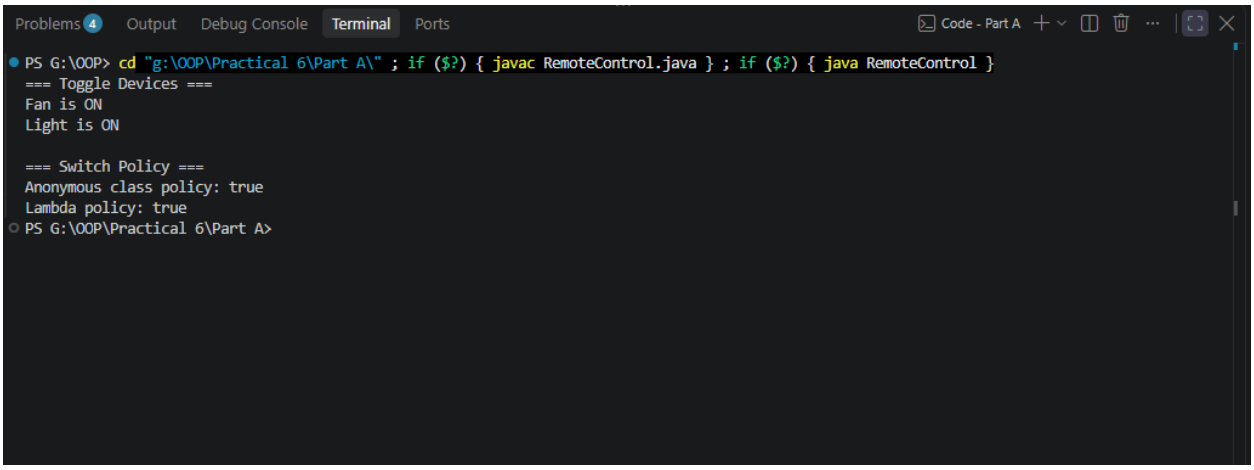
        int hour = 20;

        System.out.println(
                "Anonymous class policy: "
                + daytimePolicy.maySwitchOn(devices[0], hour)
```

```
        );

        System.out.println(
                "Lambda policy: "
                + nightPolicy.maySwitchOn(devices[1], hour)
        );
    }
}
```

Output



2. Notification senders: a functional interface Notifier with send(String); make email and SMS senders as lambdas, hold them in an array, and broadcast one message through all. Add a marker interface Urgent and send twice for senders marked urgent.

Code:

Notification.java

```
public class NotificationDemo {

    // Functional interface
    @FunctionalInterface
    interface Notifier {
        void send(String message);
    }

    // Marker interface
    interface Urgent {
    }

    // Urgent email sender
    static class UrgentEmail implements Notifier, Urgent {

        private final Notifier sender;

        UrgentEmail() {
            sender = message ->
                    System.out.println("Email: " + message);
        }

        @Override
        public void send(String message) {
            sender.send(message);
        }
    }
```

```
    // Normal SMS sender
    static class SmsSender implements Notifier {

        @Override
        public void send(String message) {
            System.out.println("SMS: " + message);
        }
    }

    // Urgent SMS sender
    static class UrgentSms implements Notifier, Urgent {

        @Override
        public void send(String message) {
            System.out.println("Urgent SMS: " + message);
        }
    }

    public static void main(String[] args) {

        // Email sender using lambda
        Notifier email = message ->
                System.out.println("Email: " + message);

        // SMS sender using lambda
        Notifier sms = message ->
                System.out.println("SMS: " + message);

        Notifier[] senders = {
            email,
            sms
        };

        String message = "Your account has been updated.";
```

```
        System.out.println("=== Normal Broadcast ===");

        for (Notifier sender : senders) {
            sender.send(message);
        }

        System.out.println();

        System.out.println("=== Urgent Senders ===");

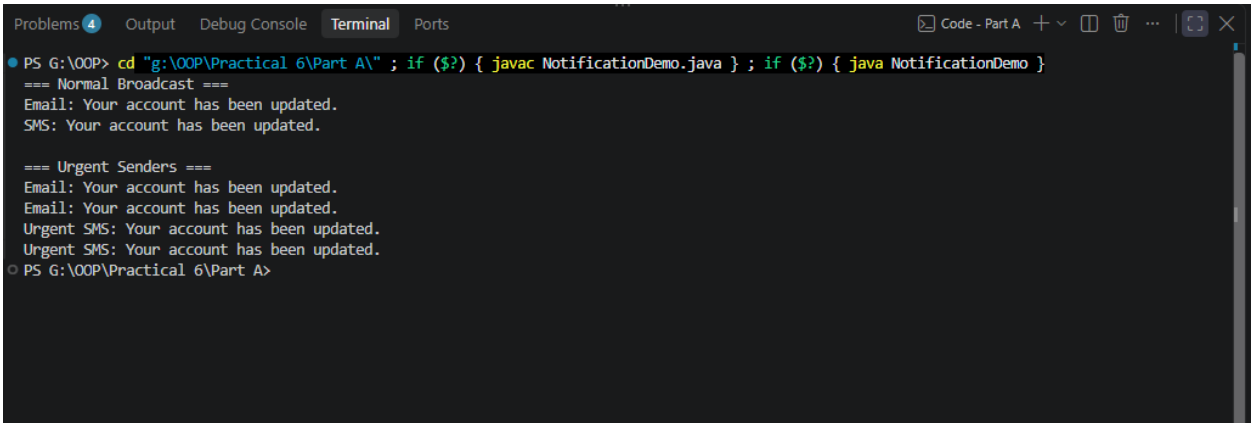
        Notifier[] urgentSenders = {
            new UrgentEmail(),
            new UrgentSms()
        };

        for (Notifier sender : urgentSenders) {

            if (sender instanceof Urgent) {

                sender.send(message);
                sender.send(message);
            } else {
                sender.send(message);
            }
        }
    }
}
```

Output:



Key Questions / Analysis / Interpretation to be evaluated during/after Implementation

1. What is the difference between an interface and an abstract class?

   Ans.

Interface	Abstract Class
Defines a **contract** that classes agree to follow.	Defines a **common base** with shared behavior/state.
A class can implement **multiple interfaces**.	A class can extend only **one class**.
Mainly used for common capabilities/behavior.	Used when subclasses have a strong parent-child relationship.
Can contain abstract, default, and static methods.	Can contain abstract and concrete methods.
Cannot have normal instance fields representing object state in the same way as a class.	Can have instance variables and constructors.
Implemented using `implements`.	Extended using `extends`.

2. What is a functional interface, and how does a lambda use it?

Ans.

A **functional interface** is an interface that contains **exactly one abstract method**.

Example:

```
@FunctionalInterface
interface SwitchPolicy {
    boolean allowed(String device, int hour);
}
```

Because it has one abstract method, we can implement it using a **lambda expression**:

```
SwitchPolicy policy =
    (device, hour) -> hour >= 8 && hour <= 20;
```

The lambda:

```
(device, hour) -> hour >= 8 && hour <= 20
```

provides the implementation of:

```
boolean allowed(String device, int hour);
```

Then:

```
System.out.println(policy.allowed("Fan", 10));
```

produces:

```
true
```

3. Why is a default method useful in an interface?

Ans.

A **default method** allows an interface to provide a method implementation that implementing classes automatically inherit.

Example from the Remote Control practical:

```
interface Switchable {

    void on();

    void off();

    default void toggle() {
        System.out.println("Toggling device");
    }
}
```

A class implementing `Switchable` does **not have to implement `toggle()`**:

```
class Fan implements Switchable {

    public void on() {
        System.out.println("Fan ON");
    }

    public void off() {
        System.out.println("Fan OFF");
    }
}
```

`Fan` automatically gets the default `toggle()` implementation.

Why is this useful?

It allows developers to **add new behavior to an existing interface without forcing every existing implementing class to implement the new method**.